

클라우드컴퓨팅

텀 프로젝트 1: 제안서

대표학생 이름	김성욱
대표학생 학번	202024191
대표학생 소속 학과/대학	정보컴퓨터공학부
분반	059

<주의사항>

- 팀 과제입니다. (팀 대표 또는 팀원 중 아무나 한 명만 제출하면 됩니다.)
- **각각의 질문 아래에 답을 작성 후 제출해 주세요.**
 - 소스코드/스크립트 등을 제출하는 경우, 해당 파일의 이름도 적어주세요.
- PLATO 제출 데드라인: **공지사항 참고**
 - 데드라인을 지나서 제출하면 0 점
 - 주말/휴일/학교행사 등 모든 날짜 카운트 함
 - 부정행위 적발 시, 원본(보여준 사람)과 복사본(베낀 사람) 모두 0 점 처리함
 - 예외 없음
- PLATO 에 아래의 파일을 제출 해 주세요
 - 보고서(**PDF 파일로 변환 후 제출, 또는 워드문서 그대로 제출**)
 - 보고서의 경우 PDF/DOC/DOCX 이외의 확장자는 제출 불가
 - 다른 확장자로 제출하였는데 채점자 컴퓨터에서 파일이 열리지 않으면 0 점 처리
 - 보고서 파일명 및 보고서 첫 페이지에 이름과 학번을 입력 해 주세요.

<개요>

이번 과제는 팀 프로젝트 제안서를 제출하는 내용입니다.

<팀 프로젝트 : 제안서>

[1] 개발 내용 소개

팀 프로젝트 주제 및 팀 프로젝트에서 개발하고자 하는 시스템/서비스/SW에 대해 소개하세요.

답변 1(프로젝트 **주제**): 아래에 한 줄로 프로젝트 주제를 정리

사용자별 Workspace와 OAuth 인증을 지원하는 OpenClaw 기반 Multi-Tenant Agent-as-a-Service 플랫폼

답변 2(프로젝트 **소개**): 아래에 한 문단으로 프로젝트를 소개

본 프로젝트는 OpenClaw를 기반으로 여러 사용자가 하나의 공유된 AI Agent 시스템을 사용할 수 있도록 하는 Multi-Tenant Agent-as-a-Service 플랫폼을 개발하는 것을 목표로 한다. 기존 OpenClaw는 에이전트를 컨테이너 단위로 격리하여 실행할 수 있는 기능을 제공한다. 그러나 기본 구조는 주로 로컬 환경에서 사용자가 직접 에이전트 시스템을 실행하고 관리하는 방식에 가깝다. 이 경우 여러 에이전트를 동시에 실행하면 로컬 컴퓨터의 메모리, CPU, 디스크 자원을 많이 사용하게 되며, 팀이나 회사 단위에서 동일한 에이전트 시스템을 반복적으로 구축해야 하는 부담이 있다.

본 프로젝트에서는 OpenClaw의 컨테이너 기반 에이전트 실행 구조를 활용하되, 이를 공용 서비스 형태로 확장한다. 즉, 에이전트 실행 환경은 서버 또는 클라우드 환경에 두고, 사용자는 자신의 Workspace와 인증 정보를 이용해 공유된 에이전트 시스템에 접근한다. 이를 통해 사용자는 로컬 컴퓨터에 무거운 에이전트 실행 환경을 직접 구성하지 않아도 되고, 팀 전체가 동일한 에이전트 시스템을 안정적으로 사용할 수 있다.

[2] 필요성 및 활용방안.

개발하고자 하는 SW의 필요성에 대해 설명하세요. 개발 결과물이 어떻게, 그리고 어떤 상황에서 사용될지(Use-Case) 설명하세요.

답변:

최근 AI Agent 시스템은 단순한 질의응답을 넘어서 코드 작성, 파일 수정, 명령 실행, 테스트 수행 등 실제 작업 환경에 가까운 기능을 수행하고 있다. 이러한 Agent 시스템은 강력하지만, 로컬 환경에서 실행할 경우 많은 컴퓨팅 자원을 요구한다. 특히 여러 개의 에이전트를 동시에 실행하거나, 각 에이전트가 독립적인 도구 실행 환경을 필요로 하는 경우 메모리와 CPU 사용량이 크게 증가한다. 개인 노트북이나 일반 데스크톱 환경에서는 이러한 시스템을 장시간 안정적으로 운용하기 어렵다.

또한 팀 단위로 AI Agent를 활용하려는 경우, 각 팀원이 동일한 에이전트 시스템을 개별적으로 설치하고 설정해야 하는 문제가 발생한다. 에이전트 설정, Docker 환경, 모델 인증 정보, 실행 도구 구성이 사용자마다 달라지면 재현성이 떨어지고 운영 관리도 어려워진다. 따라서 에이전트 시스템 자체는 공용 인프라로 제공하고, 사용자는 자신의 Workspace와 인증 정보를 통해 서비스를 사용하는 구조가 필요하다.

본 프로젝트에서 제안하는 Agent-as-a-Service 구조는 이러한 문제를 해결할 수 있다. 에이전트 실행은 서버 또는 클라우드 환경에서 수행되므로 사용자의 로컬 컴퓨터 자원 부담이 줄어든다. 또한 에이전트 시스템을 한 번 구축하면 여러 사용자가 공통으로 사용할 수 있어 팀 단위 운영에 적합하다. 컨테이너 단위로 에이전트 실행 환경을 분리하므로 사용자별 작업 격리와 실행 환경 재현성도 확보할 수 있다. 나아가 컨테이너 실행 시간과 자원 사용량을 기준으로 사용량을 측정하면, 향후 종량제 과금이나 내부 비용 산정에도 활용할 수 있다.

[3] 선행기술 조사

개발하고자 하는 SW/시스템과 동일한 또는 유사한 기술이 이미 개발되었는지를 조사하세요. 관련 제품을 조사한 결과를 첨부하거나, 관련 논문/특허를 검색한 결과를 첨부해도 됩니다. 관련 제품/기술에 대한 간략한 소개 및 출처도 작성하세요.

답변:

OpenClaw 는 AI Agent 를 실행하고 관리하기 위한 오픈소스 프로젝트로, 에이전트 실행 환경을 컨테이너 기반으로 격리할 수 있는 기능을 제공한다. 이를 통해 에이전트가 실행하는 명령이나 도구 작업이 호스트 환경에 직접 영향을 주지 않도록 제한할 수 있으며, 에이전트별로 독립된 실행 환경을 구성할 수 있다. 또한 OpenAI 호환 API 형태의 Gateway 기능을 제공하여 외부 클라이언트가 에이전트에 요청을 보낼 수 있는 구조도 갖추고 있다.

하지만 OpenClaw 의 기본 Gateway 는 사용자별 권한 모델이나 Multi-Tenant 서비스 운영을 전제로 설계된 구조는 아니다. OpenAI 호환 API 를 통해 특정 에이전트에 요청을 보낼 수는 있지만, 이를 그대로 공용 서비스로 노출하면 사용자별 Workspace 접근 제어, 사용자별 OAuth 인증 분리, 요청량 제한, 감사 로그 관리 등이 부족할 수 있다. 따라서 OpenClaw 를 단순히 그대로 사용하는 것만으로는 회사나 팀 단위의 Agent-as-a-Service 플랫폼을 구성하기 어렵다.

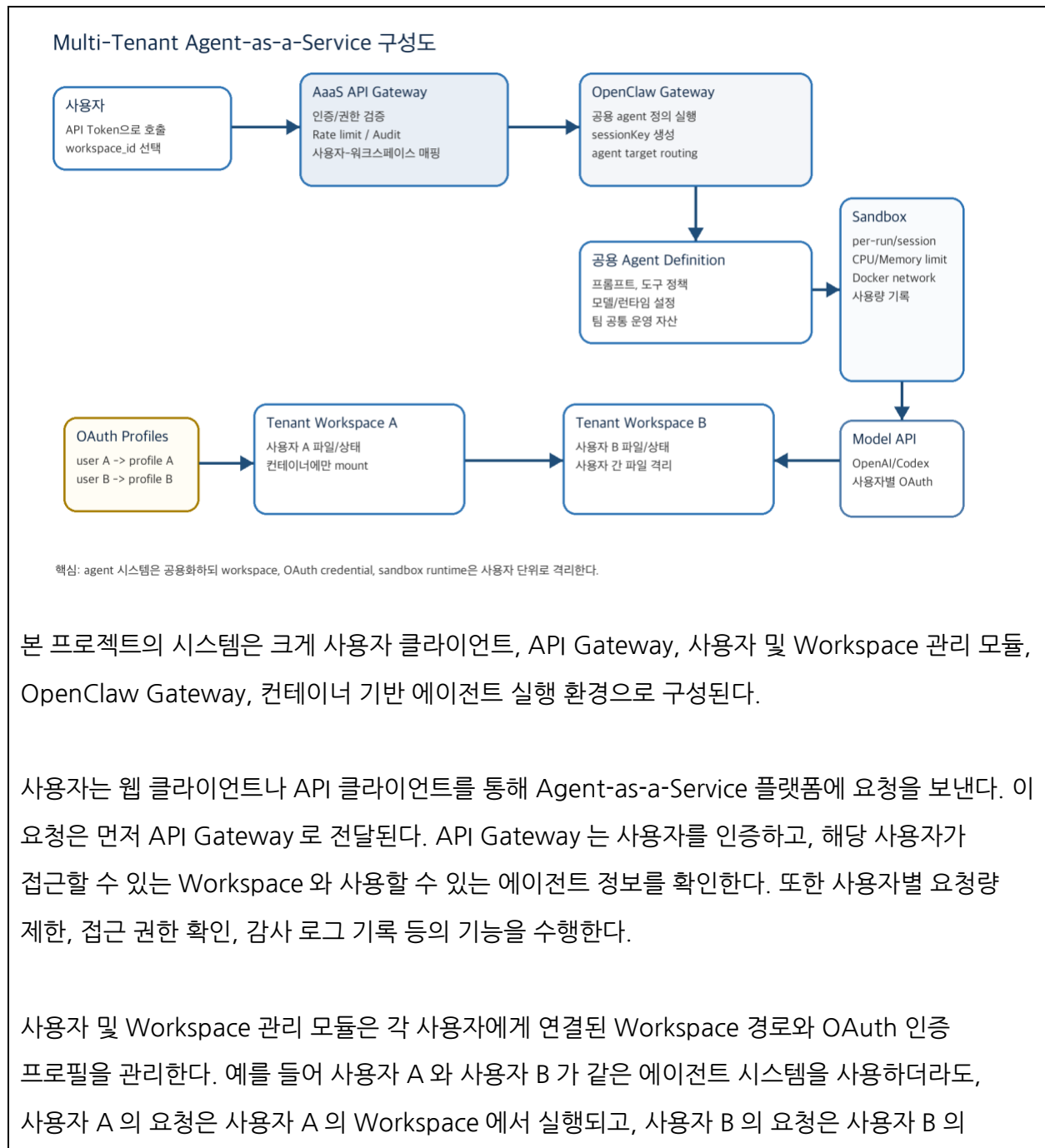
Docker 는 본 프로젝트의 핵심 기반 기술이다. 기존 OpenClaw 는 Docker 컨테이너를 이용해 에이전트 실행 환경을 격리한다. 본 프로젝트에서는 이 기능을 확장하여 사용자별 작업 공간을 컨테이너에 안전하게 연결하고, 컨테이너 실행 단위로 자원 사용량을 추적하며, 여러 사용자의 요청이 동시에 들어와도 독립적인 실행 환경을 유지하도록 한다. 즉, 단순히 컨테이너 안에서 에이전트를 실행하는 수준을 넘어, 컨테이너를 서비스 운영 단위로 활용하는 것이 본 프로젝트의 차별점이다.

[4] 제안하는 SW/시스템의 구성도

최종 결과물의 구성 및 동작방식을 다이어그램을 그려서 설명하세요. 예를 들어, [WAS]+[DB]로 구성된 SW 인 경우, [사용자], [WAS], [DB]를 나타내는 다이어그램을 각각 그리고 [사용자]가 [WAS]로 어떤 요청을 보내는지, [WAS]가 사용자에게 어떤 답변을 보내는지, [WAS]와 [DB]사이에

어떤 요청/응답이 오가는지를 설명하는 다이어그램을 그리세요. 그림에 텍스트를 충분히 추가하여 어떤 데이터/명령이 오고 가는지, 어떻게 상호작용 하는지를 나타내세요.

답변:



Workspace 에서 실행된다. 마찬가지로 모델 접근에 필요한 OpenAI 또는 Codex OAuth 인증 정보도 사용자별로 분리된다. 사용자가 직접 인증 프로필 ID 를 지정하는 방식은 보안상 위험할 수 있으므로, 서버가 로그인된 사용자 정보를 기준으로 내부 인증 프로필을 매핑한다.

OpenClaw Gateway 는 실제 에이전트 요청을 처리하는 계층이다. API Gateway 에서 검증된 요청은 OpenClaw Gateway 로 전달되고, OpenClaw 는 지정된 에이전트를 실행한다. 이때 에이전트는 컨테이너 내부에서 동작하며, 컨테이너에는 해당 사용자에게 할당된 Workspace 가 연결된다. 에이전트가 파일을 읽거나 수정하거나 명령을 실행하더라도, 작업 범위는 해당 사용자의 Workspace 로 제한된다.

컨테이너 실행 환경은 에이전트 실행 도구와 작업 환경을 포함한다. 각 에이전트 실행은 독립된 컨테이너에서 수행되므로 실행 환경 충돌을 줄일 수 있고, 필요에 따라 컨테이너를 생성하거나 종료할 수 있다. 또한 컨테이너 실행 시간, CPU 사용량, 메모리 사용량 등을 측정하면 사용자별 사용량 분석이나 비용 산정에도 활용할 수 있다.

기대효과

본 프로젝트를 통해 로컬 실행 중심의 AI Agent 시스템을 공유 가능한 Agent-as-a-Service 구조로 확장할 수 있다. 사용자는 자신의 컴퓨터에 무거운 에이전트 실행 환경을 직접 구성하지 않아도 되며, 서버에서 제공하는 공용 에이전트 시스템을 사용할 수 있다. 이로 인해 로컬 컴퓨팅 자원 사용량이 줄어들고, 팀 단위로 동일한 에이전트 환경을 안정적으로 공유할 수 있다.

또한 사용자별 Workspace 와 OAuth 인증 정보를 분리함으로써 Multi-Tenant 환경에서 필요한 기본적인 격리 구조를 제공할 수 있다. 같은 에이전트 시스템을 사용하더라도 각 사용자의 작업 파일과 인증 정보는 독립적으로 관리되므로, 팀이나 회사 내부에서 공용 AI Agent 서비스를 운영하는 기반이 될 수 있다.

컨테이너 기반 실행 구조를 활용하면 에이전트 실행 환경의 재현성과 격리성을 확보할 수 있다. 컨테이너는 필요할 때 실행하고 작업이 끝나면 종료할 수 있으므로, 향후 클라우드 환경으로 확장할 경우 사용한 만큼 비용을 지불하는 종량제 구조와도 잘 맞는다. 따라서 본 프로젝트는 단순한

OpenClaw 기능 사용을 넘어, 클라우드 컴퓨팅 관점에서 AI Agent 실행 환경을 서비스화하는
실험이라는 의미를 가진다.

[5] 동작 환경

최종 결과물이 클라우드, PC/노트북, 모바일 기기(스마트폰 등), 라즈베리파이 등 어느 플랫폼에서 동작하는 지에 대한 설명도 추가하세요.

답변:

본 프로젝트는 로컬 개발 환경에서 Docker 를 이용해 구현하고 검증한다. 클라우드 배포는 최종 확장 가능성으로 고려하되, 프로젝트 범위에서는 로컬 머신 또는 로컬 서버에서 컨테이너 기반 Agent-as-a-Service 구조를 재현하는 것을 우선 목표로 한다. 개발 환경은 Docker 와 Node.js 기반의 OpenClaw 실행 환경을 사용한다.

에이전트 실행은 Docker 컨테이너 내부에서 수행되며, 사용자의 Workspace 는 컨테이너에 마운트되는 방식으로 연결된다. API Gateway 와 OpenClaw Gateway 는 로컬 네트워크 환경에서 연동한다. 모델 호출은 OpenAI 또는 Codex OAuth 인증 정보를 사용한다.

최종 결과물은 클라우드 서비스에서 컨테이너 단위 배포가 가능한 서비스를 활용하여 동작하는 것을 목표로 하되, 세부적으로 어떤 서비스를 활용하는지는 개발 현황에 맞춰 결정하는 것으로 한다.

끝! 수고하셨습니다 ☺